

SPM: Intro Lecture

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Why parallel computing?
- Moore's law and implications of parallelism on power consumption
- A glimpse into heterogeneous multi-cores
- Top500 and Green500 lists
- Software challenges

Key message: performance, energy efficiency, and scalability are software responsibilities

What is parallel computing?

- **Parallel computing** is the practice of using multiple processors in parallel to solve problems faster than with a single processor or tackle larger problem size
 - Parallel computing is not only about speedup, but about making problems feasible
- Core capabilities:
 - identifying and exposing parallelism in algorithms and software systems
 - Finding parts of a problem that can run at the same time
 - Finding dependencies between different parts
 - Evaluating trade-offs
 - Understanding the costs, benefits, and limitations of a chosen parallel approach
 - Communication overhead, scalability and resource usage

Parallel Machines

- Parallel machines (examples):
 - **Compute Cluster**: multiple independent computers connected with one or more high-speed networks (Gigabit Ethernet, Infiniband, Omnipath, ...)
 - **Beowulf compute cluster** made by «inexpensive» PCs connected to a small LAN usually Linux-based
 - **SMP (Symmetric Multi-Processor)**: multiple processor chips connected to a shared memory hierarchy
 - **CMP (Chip Multi-Processor aka Multicore)**: multiple compute units (called cores) packed on a single chip

These architectures lead to **different programming models**, which we will study in this course



Why Parallel Computing?

- Historically, the main motivation was to **improve performance** and **tackle larger problems**
 - Mostly used in computational science and engineering for simulation
 - Mainly for problems too large to be solved on a single computer
- For a long time, parallel computing was **considered a niche area in computer science**
- HPC Application Areas (some):
 - Weather Forecasting and Climate Simulation
 - Flood simulation
 - Computational Chemistry
 - Bioinformatics (DNA pattern matching)
 - Earthquake engineering
 - Molecular Modelling
 - Material Science
 - Computational Fluid Dynamics
 - Aerodynamics (Air Flow Simulations)
 - Oil and Gas Simulation

Why Parallel Computing Today?

- Around 2004-2005, the industry shifted toward multiple processors (multicore) in all systems (laptops, desktops, workstations, servers, and then mobile phones, embedded devices, etc...)
 - Today's off-the-shelf consumer-grade server machines (often equipped with powerful GPUs) have computing power comparable to supercomputers from two decades ago!
- Parallel Computing is no longer the domain of only parallel-enthusiastic system experts.
- It is now **a fundamental background** of any computer scientist and practitioner
 - Parallelism is no longer optional, it is the default execution model

Why Parallel Computing today? (cont.)

- **AI revolution**

- The rise of AI, particularly Deep Learning, **has amplified the demand for parallel computing**. Modern AI algorithms rely on the parallel processing power of GPUs and TPUs for training and inference of complex models

- **Parallelism in AI:**

- Training AI models involves billions of operations, requiring highly parallelized hardware and frameworks (e.g., CUDA, TensorFlow, PyTorch).
- AI applications, image recognition, natural language processing, and real-time recommendation systems leverage parallelism for fast inference
- **Many AI workloads map naturally to structured parallel patterns (e.g., data parallelism)**

Why Parallel Computing Today? (cont.)

- **Big Data Analytics (BDAs)**
 - Very large datasets (Terabyte, Petabyte, Exabyte)
 - Real-time Data Analytics/Data Stream Processing (e.g. high-frequency financial trading)
- A single multicore server cannot provide the memory capacity, computational power, and I/O bandwidth needed for very large BDAs
 - The software stack and programming models used in traditional HPC applications and BDA applications are not the same (e.g., Fortran/C vs Java/Python, MPI/OpenMP vs Spark/Hadoop, ...)
- **HPC for AI**
 - Massive use of powerful GPUs for training phases (and also for the inference phase)
 - HPC environments provide optimized frameworks for AI (TensorFlow, PyTorch, ...)
- The boundary between HPC, Big Data, and AI is increasingly blurred
 - Solving modern AI problems requires combining all three domains

Why Parallel Computing Today? (cont.)

- **Cloud-Edge compute continuum**

- Latency, bandwidth, privacy, and reliability push compute from cloud toward devices and near-data edge
- Applications are partitioned across (fog) devices → edge → cloud (**heterogeneity**)
- You must design parallel programs that place work where data lives and orchestrate many nodes/accelerators

- **Quantum computing**

- Domain-specific accelerator inside HPC (**hybrid** quantum-classical workflows)
- Orchestration and scheduling of many quantum jobs within classical workflows
 - Offloading of subproblems to QPU, while CPU/GPU host is used for orchestration

From Motivations to Technology

- **We've seen WHY parallel computing matters**
- **Now let's understand HOW hardware evolved**

Technology trends and Moore's Law

- **Gordon Moore** (co-founder of Intel) predicted in 1965 that the transistor density of semiconductor chips would double roughly every 18 months
 - 2X transistors/chip every 1.5 years (so-called «**Moore's law**»)
- It was possible to integrate on a single chip more and more transistors with technological improvements over the years
 - microprocessor chips have become smaller, denser, and more powerful over time

NOTE: Moore's law refers to transistor count, not to performance

Implications of the Moore's law

- The cost of processor chips and memories has decreased as well
- Hardware improvements that drove performance (pre-2024). Why?
 - Smaller and faster transistors
 - Improved microarchitecture (i.e., with more logic), thus more instructions per clock cycles (i.e., more HW parallelism!)
 - Reduced gate delays per cycle, higher clock frequency
 - Larger integrated caches to reduce the Von Neuman bottleneck
 - SIMD instructions (Instruction Level Parallelism – ILP)
- For decades, hardware improvements automatically translated into higher performance for sequential programs.
- Common view: «**Why bother with parallel programming for performance? Just wait one or two years!**»
- This was called «**The free-lunch Era**»

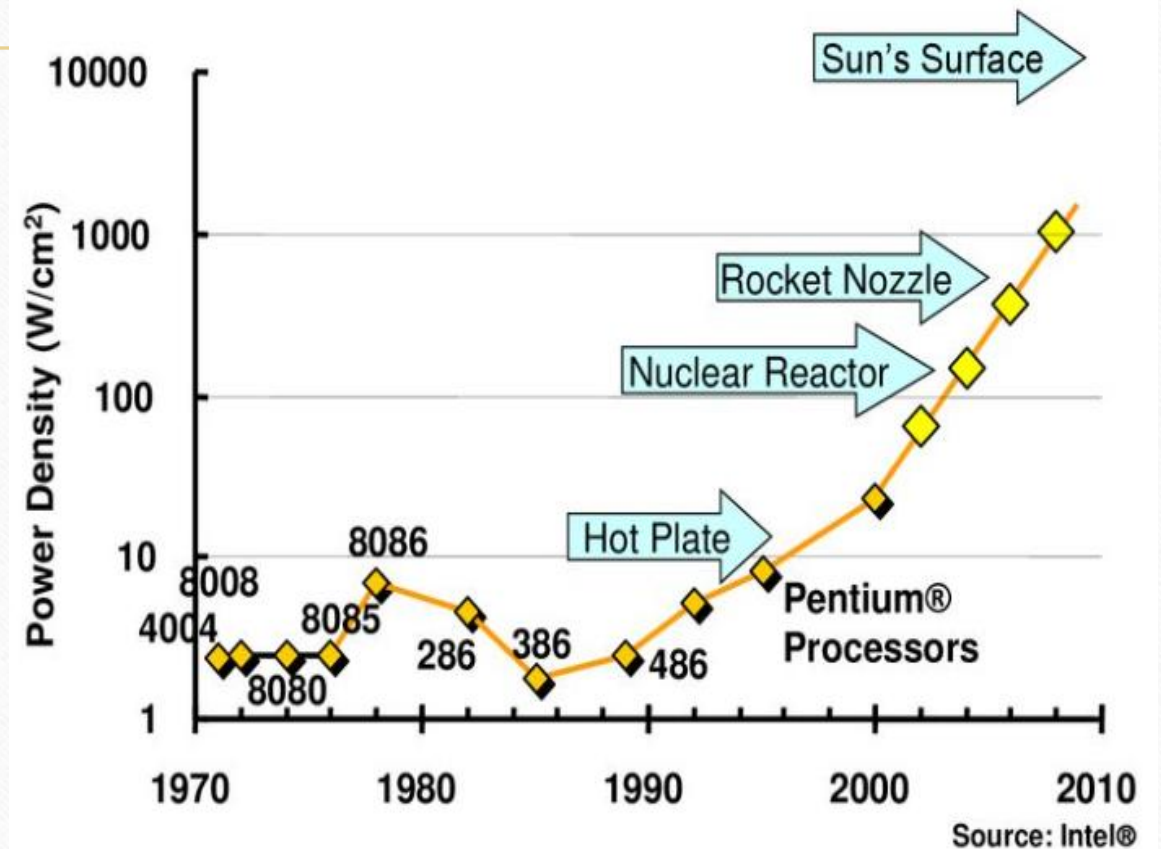
Limits of Single-Chip improvements

- The primary limits of single-chip performance improvement are related to packing more and more transistors into the chip
 - Speed of light limit (fundamental insurmountable limit)
 - At 3GHz, signals travel only ~10cm per clock cycle. Chip size becomes a limiting factor
 - Power density (power was increasing over the years!)
- «**Dennard's scaling law**». Dennard's 1974 observation was that power density was constant as transistors got smaller.
 - That was true for almost 30 years!
 - Dennard scaling ended in 2004-2005, not because transistors didn't continue to get smaller (i.e., Moore's law still applies!) but for power issues (the so-called **power wall**)

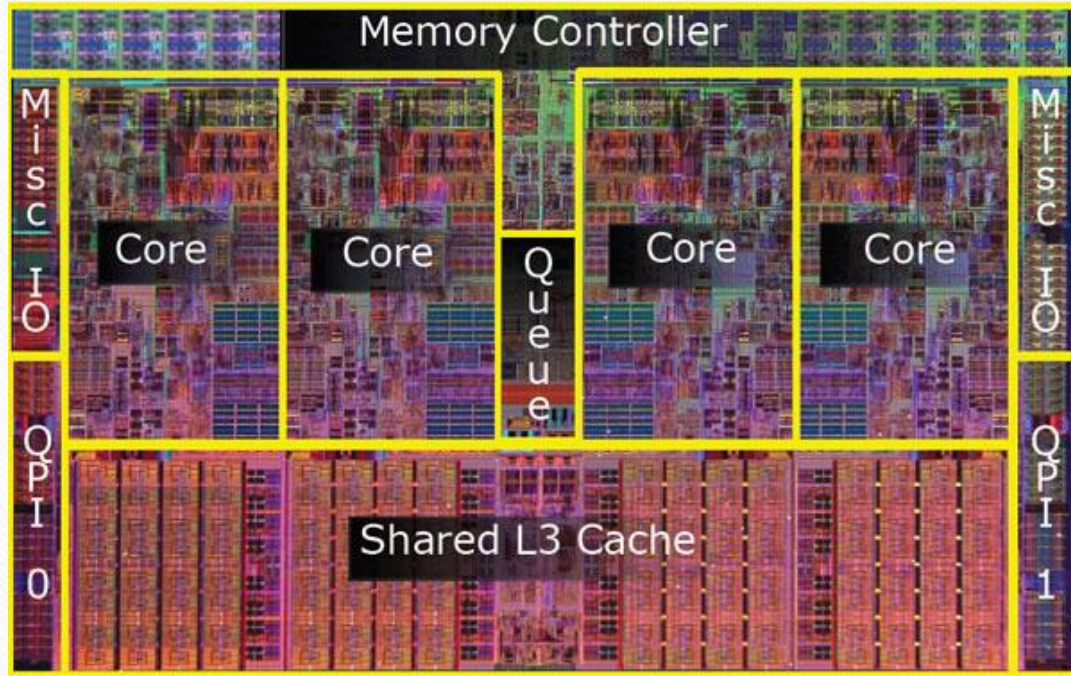
Power density

- In the early '00 Intel VP Patrick Gelsinger :
«business as usual will **not** work in the future»

He warned that, if clock speed scaling continues at its present pace, by 2005, high-speed processors would have the power density of a nuclear reactor, by 2010, a rocket nozzle, and by 2015, the sun's surface.



Moving to Multicore (Chip Multi-Processor)



- Intel's ® Nehalem processor
- 2008-2010
- From 1.06 GHz to 3.33 GHz
- 4-core with two QPI (Quick Path Interlink)
- Private L1 and L2 and large shared L3 cache

- Processor manufacturers put many processing cores on the microprocessor chip
 - Each generation of Moore's Law potentially increases (doubles) the number of cores

Moving to CMP to reduce power

- Use explicit parallelism to reduce power consumption

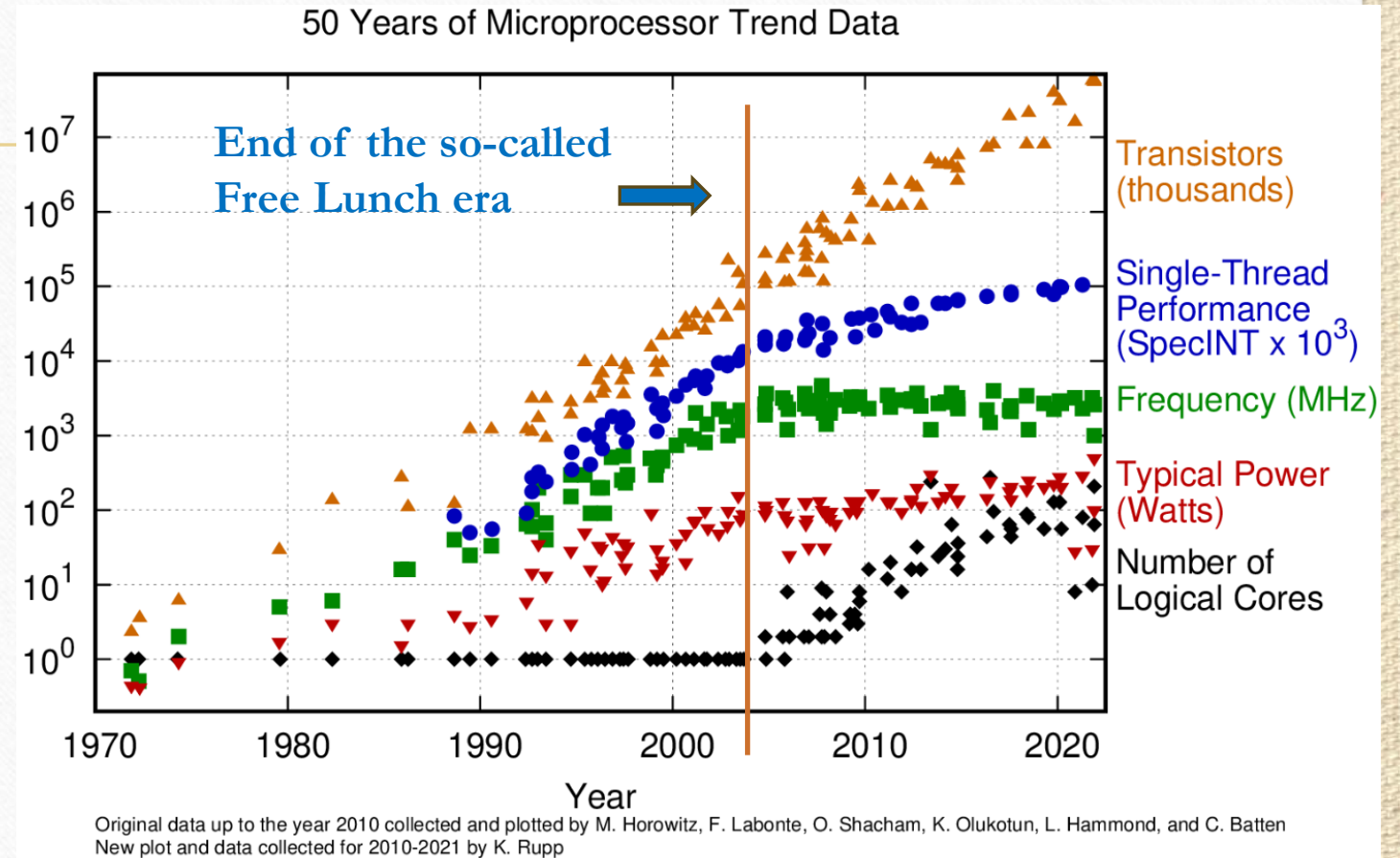
$$Power_{dynamic} \approx \alpha \cdot C \cdot V^2 \cdot F$$

$$Perf = Ncores \cdot F$$

- V= voltage , C= capacitance, F= clock frequency
- But $V \propto F$ therefore $Power_{dynamic} \propto C \cdot F^3$
- If we double $Ncores$, we double $Perf$ but also $Power$
- If we double $Ncores$, and we halve V and F we have the same $Perf$ but **the power is reduced by a factor of four** (i.e., $2 \cdot \alpha \cdot C \cdot \left(\frac{V}{2}\right)^2 \cdot \left(\frac{F}{2}\right) = \frac{Power_{dynamic}}{4}$)

CMPs Trend

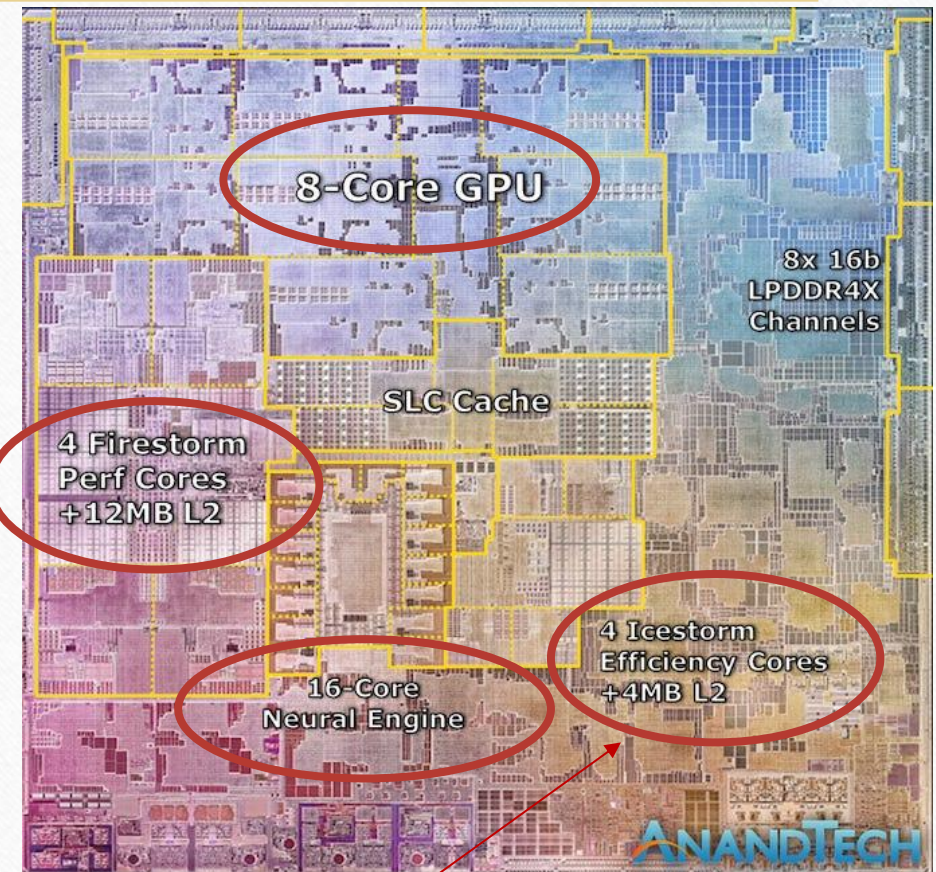
- Moore's Law (transistor growth) kept going
 - Moore's law is still valid if we account for the number of cores
- Dennard scaling broke around 2004
 - Frequency stops increasing
 - Single thread performance is increasing quite slowly
 - **Dark silicon**: not all transistors can be powered on simultaneously due to power constraints
- Large cache units integrated within the chip



Source: <https://github.com/karlrupp/microprocessor-trend-data>

Heterogeneous Multicore

- Heterogeneous CMPs integrate different types of processor cores on a single chip (**asymmetry**)
- Core types: CPU, GPU, NPU, DSP
- Power Efficiency: assign different workloads to the appropriate core type to minimize power consumption
- Leveraging specific cores to maximize system performance (specialization)
- Example:
 - **ARM big.LITTLE** for mobile and embedded devices
 - **Apple M1** (picture): 8-core GPU, 4 “big” + 4 “small” cores, NPU
- Heterogeneity is now the norm, not the exception



«big» cores

«small» cores

Supercomputers



- A supercomputer is like a F1 Ferrari
- They are large-scale parallel systems with strict energy constraints
- A supercomputer is a highly advanced and powerful computing system designed to perform massive computations at high speeds. It is designed for raw performance. Current target: **Exascale Computing** ($O(10^{18})$ FLOPS)
 - Examples: Frontier (US), Fugaku (Japan), LUMI (Finland), Leonardo (Italy)
- Multiple independent CMP nodes (often with some powerful GPUs) connected with one or more high-bandwidth, low-latency networks and with a dedicated network for I/O offering high-bandwidth.

Top500 trend

- Which are currently the fastest computers in the world?
- Supercomputers are ranked by their maximum **LINPACK** benchmark performance (<https://www.top500.org>)
 - Measures the performance for solving a dense system of linear equations ($A \cdot x = b$) in terms of FLOPs
 - Top-ranked system in June 2025 was **El Capitan**
 - Hewlett Packard Enterprise (HPE)
 - $R_{max} = 1.742$ EXAFLOP/s (*) ($R_{peak} = 2.746$ EXAFLOP/s)
 - 11,039,616 cores (CPUs + Accelerators)
 - ~30MW peak power consumption
 - OS: TOSS (Linux-based)

- **LINPACK measures peak capability, not application efficiency**

(*) 1 exaflop = 1 billion billion (i.e., 10^{18}) floating point operations per second (FLOPs)



Green500



- Ranks supercomputers based on **FLOPS-per-Watt** of achieved LINPACK performance
 - Top-ranked system in November 2023 was **JEDI**
 - ParTec/EVIDEN
 - 72.733 GFLOPS/Watts $R_{max} = 4.504$ PFLOPS)
 - 19,584 cores (CPUs + Accelerators)
 - #259 in the Top500 list
 - OS: RedHat Linux and Modular Operating System
 - **El Capitan** (i.e., #1 in Top500) was ranked 25



<https://www.top500.org/news/energy-efficient-supercomputing-comes-of-age-with-top500-green500-merge/>

Rank	TOP500 Rank	Computer	Manuf	Country	Total Cores	Rpeak [TFlop/s]	Power (kW)	Energy Efficiency [GFlops/Watts]
1	259	BullSequana XH3000, Grace Hopper Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200	ParTec/EVIDEN	Germany	19.584	5.131,78	67,31	72,73
2	148	BullSequana XH3000, Grace Hopper Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, Red Hat Enterprise Linux	EVIDEN	France	47.328	16.322,59	160,16	70,91
3	484	HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, RHEL	HPE	France	16.128	4.056,40	36,60	69,10
4	183	HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11	HPE	UK	34.272	9.285,90	117,08	68,83
5	255	ThinkSystem SD665-N V3, AMD EPYC 9655 96C 2.6GHz, NVIDIA H100 SXM5 80GB, Infiniband NDR, Rocky Linux 9.4	Lenovo	Germany	19.440	7.451,65	-	68,18

From Hardware to Software

Hardware improved a lot over the years, what about Software?

Let's explore the software challenges

CMPs, are they enough?

- «The switch to multiple processors per chip around 2005 did not come from some breakthrough that dramatically simplified parallel programming or made it easy to build multicore computers. The change occurred because there was no other option due to the ILP walls and power walls»
- «... **performance is now a programmer's burden**. ... If programmers want their programs to go faster with each generation, **they must make their programs more parallel**. The popular version of Moore's law – increasing performance with each generation of technology – **is now up to programmers**»

J. L. Hennessy, D. A. Patterson «Computer Architecture – A Quantitative Approach» 6th Edition, Morgan Kaufmann

Answer to the title question: «**No, they aren't**».

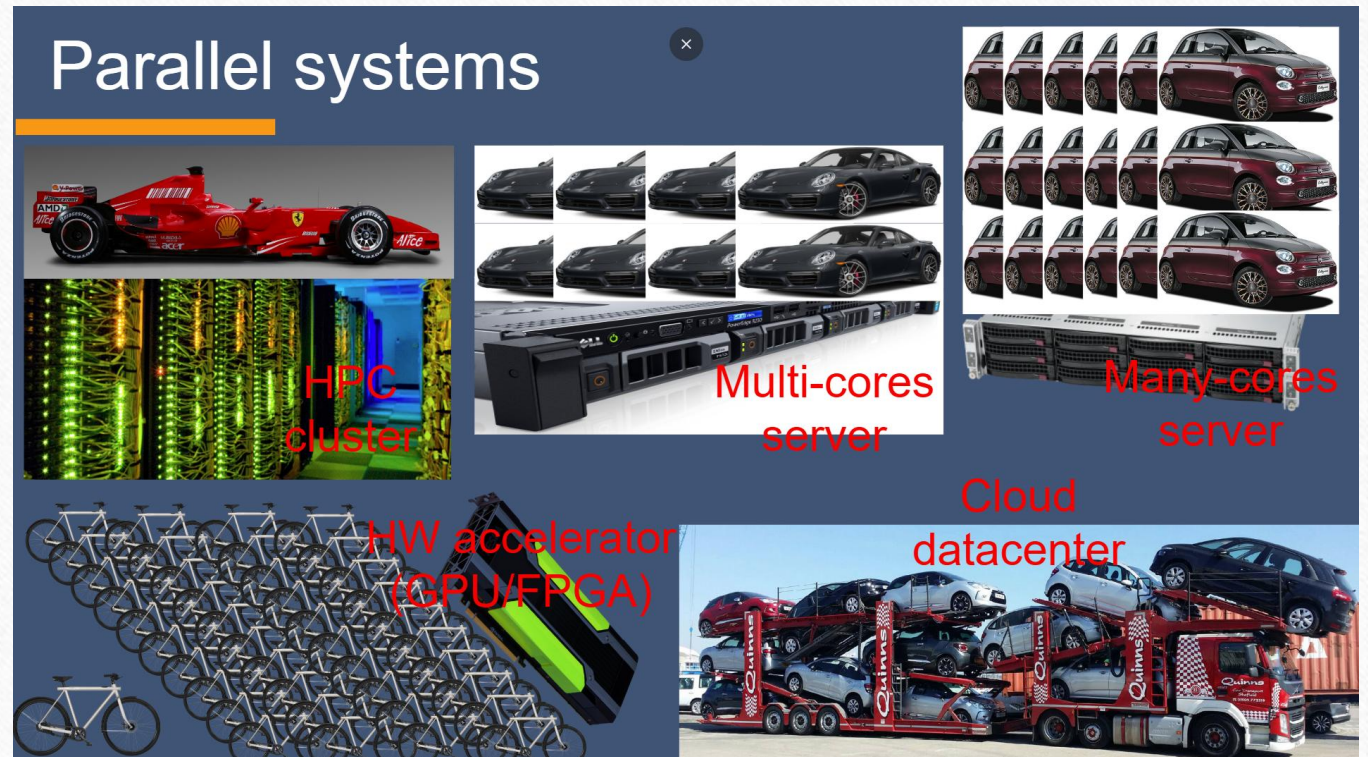
Additionally, **performance portability**, i.e., the challenge of achieving good performance across architectures without rewriting code, became a central problem

Software Challenges

- **The shift to CMPs and thus to parallel computing was not inexpensive for software!**
 - The transition introduced new complexities (dealing with memory consistency, cache coherence, etc...), leading to an initial spike in the bug count due to issues such as concurrency problems, race conditions, and thread synchronization errors.
Several software programs had to be rewritten
- While some complexities related to parallelism may eventually be hidden in libraries, compilers, high-level languages for parallel programming, etc....
 - **... much work remains to be done!**

Software Challenges (cont.)

- How can we write software to utilize HW parallelism efficiently?
- Lots of HW complexities:
 - On-chip complexity
 - Multiple caches
 - Hyperthreading/SMT
 - Vector units & integrated GPU
 - Heterogeneous cores
 - System-level complexity
 - Heterogeneous cores
 - Accelerators
 - Scale-out complexity
 - Clusters of heterogeneous nodes
 - Cloud datacenter



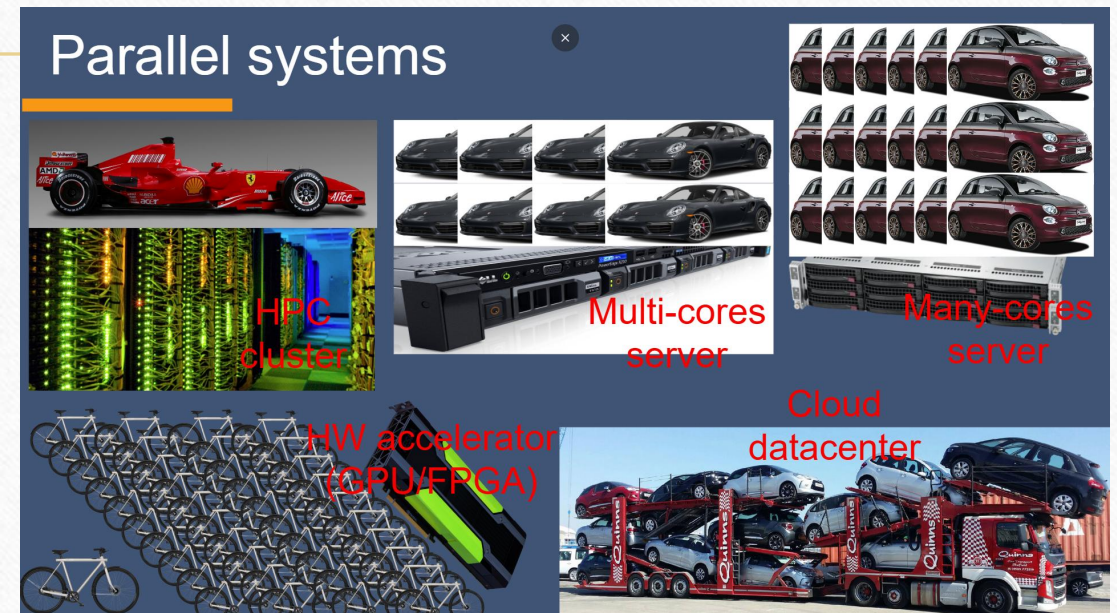
Different Software needs

- **Different needs in different contexts**

- HPC: absolute performance
- IoT: low power consumption
- Financial: low-latency
- Industry applications: real-time
- Cloud: horizontal scalability
-

- **What if we want to move our SW from one platform to a different one?**

- **In many cases, SW must be (at least partially) rewritten.**
- **The SW engineering effort is still too high!**



Open questions in Software development

- **(Some) Open questions:**
 - How can we design programming models, languages, and tools that make parallel programming accessible to non-experts?
 - How can we design parallel programs for performance and energy efficiency?
 - How can we address **performance portability** (from a “Ferrari” to a pool of “Bicycles”)?
 - How do we manage workloads with irregular, dynamic, or unpredictable computation and communication patterns?
 - How can we efficiently program and manage systems with diverse components, such as CPUs, GPUs, FPGAs, and TPUs?

Key Takeaways

- **Hardware perspective**
 - Moore's Law still holds
 - Dennard scaling ended around 2005
 - Multicore is the solution to the power wall
 - Heterogeneity is now the norm
- **Software perspective**
 - Performance is now the programmer's responsibility
 - Performance portability is a challenge
 - Energy efficiency requires careful software design

Suggested readings

- «**The Free Lunch Is Over – A Fundamental Turn Toward Concurrency in Software**» by Herb Sutter
<http://www.gotw.ca/publications/concurrency-ddj.htm>
- «**Myths and Legends in High-Performance Computing**» S. Matsuoka, J. Domke, M. Wahib, A. Drozd, T. Hoefler <https://arxiv.org/abs/2301.02432>
- Chapter 1, Section 1.3 «**HPC Trends and Rankings**» of the course book (Parallel Programming Concepts and Practice)
- «**Supercomputing for Artificial Intelligence**» <https://jorditorresbcn.github.io/supercomputing-for-ai-book/>